FRED TALKS

25th July 2026

CONTENTS

Test Coverage Won't Save You
JENN COOPER · 1991 WORDS

Scaling Managed Agents: Decoupling the brain from the hands
ANTHROPIC ENGINEERING BLOG · 2024 WORDS

Control the ideas, not the code
ANTIREZ.COM · 1245 WORDS

Giving LLMs a personality is just good engineering
SEANGOEDECKE.COM RSS FEED · 1062 WORDS

Your Agent is a Distributed System (and fails like one)
MAHESH'S BLOG · 1053 WORDS

Test Coverage Won't Save You

JENN COOPER · 09 JUN 2026 · SOURCE

A year ago, if a codebase accumulated three different ways to do the same thing, somebody would usually notice.

A reviewer might leave a comment. A senior engineer would mutter something about "yet another helper," and eventually the team would clean it up. Maybe they'd write a short doc. People would learn, collectively, that this was not the way.

This was not a perfect system. Human review can be slow. People miss things. Pattern docs get stale. Engineers develop opinions that are 70% wisdom and 30% scar tissue.

Still, the friction did something useful. It kept codebases from drifting too quickly.

Coding agents change that.

"Now, this is a room with electricity. But it has too much electricity."

Once agents are writing a meaningful share of code, the question is no longer just "can we get working code faster?" Obviously, yes. We can. Sometimes hilariously so.

The more interesting question is: what does the codebase teach the next agent?

Because it does teach.

Every merged PR becomes a precedent. If the repository contains one clear pattern, the next agent has a decent shot at following it. If it contains three slightly different patterns, the next agent may extend one, combine two, or invent a fourth for reasons that are, in technical terms, vibes.

The funhouse builds itself

Coding agents, like some of us, have a chaotic inner goth teenager.

Their non-deterministic nature means they can be inconsistent, often in surprising ways. They can generate the weird little leap that solves the problem, or they can generate novelty where nobody asked for it.

The dangerous thing is that most of the code is fine, if you look at it in isolation.

A PR passes tests. The implementation makes sense. The file is clean enough. You look at the diff and think, yeah, sure, that seems okay.

But zoom out, and the codebase as a whole has started to get weird.

- three near-duplicate utilities
- multiple data-fetching patterns
- parallel abstractions solving almost the same problem

The codebase remains locally **cohesive**, while slowly losing global **coherence**.

That matters more than it used to, because even more than humans, agents are responsive to the signals around them. A clear codebase gives them clear precedent. A muddled codebase gives them confusion.

You can put principles in `CLAUDE.md` or docs, but they're... advisory. They compete with task instructions and inline comments for attention, get summarized when context windows compact, and depend on the agent attending to the right instruction at the right moment.

If your codebase contains three slightly different ways to solve a problem, the next agent has to infer which one is canonical.

From the agent's point of view, this is not irrational. The repository gave it mixed evidence.

The scary outcome is not code that obviously fails. It is code that superficially keeps working, while the shape of the system increasingly gets worse.

Tests are nice

Most discussions about AI-native development jump from this problem – agents' tendency to accumulate tech debt – directly to tests.

And yes, across the industry, teams are writing dramatically more tests than they ever have. Agents have made high test coverage affordable in a way it never used to be.

Recently, Garry Tan argued that the primary way to keep AI agents on track is 90% test coverage:

> *Tests are the ratchet. 90% coverage, every PR, no exceptions.*

And indeed, test coverage is the simplest kind of ratchet: a mechanism that allows motion in one direction only, like a socket wrench that turns the bolt forward but never lets it spin back. Once a test locks in a behaviour, it becomes difficult to accidentally regress.

But it's worth being specific about what tests actually do.

Unit tests check that a function still returns what it returned before. Integration tests check that pieces still wire together the same way. E2E tests reach further: they check whether the product still does what it used to, and the assertions you prioritize there should be an important act of human judgment.

But they all share the same basic shape: tests verify that code does what it did before.

Whether what it did was even the *right way to do it* is a separate question.

Tests ratchet behavioural sameness.

Meanwhile docs and evals can pin down reasoning and behaviour bars, which is sometimes more important. But none of these is directly looking at the *shape* of the system.

90% coverage of good patterns

A coverage ratchet only improves the codebase if the patterns it's locking in are already good ones.

If a questionable pattern already exists in the code, the next agent is more likely to extend it. The tests generated around that implementation only reinforce it further.

Over time, high coverage can unintentionally fossilize architectural drift, rather than prevent it in the first place.

At first, especially during prototyping, this can be easy to miss, because nothing looks obviously broken. Coverage stays high. The system works. But over time, certain parts of the codebase become strangely resistant to cleanup.

Tests that protect the wrong behaviour create drag against coherence. The harder you push for coverage, the more deeply you can lock in the shape you happened to start with.

This isn't a criticism of tests. Tests are necessary. But tests primarily validate behaviour, while many of the emerging failure modes in agent-generated systems, especially as they grow, are failures of *shape*.

"Uh, I wouldn't take it down if I were you. It's a load-bearing poster."

Fitness functions protect the shape of the system

Unit tests ask whether the code still does what it did before. Fitness functions ask a different question: is the system still shaped the way we want?

Most codebases already have a few simple versions of this. Linters, type checks, and import-boundary rules all encode some idea of acceptable shape, at the micro level.

But in agentically-maintained codebases, we need an additional kind of macro fitness function. It has a couple of important jobs:

- catch bad agent choices at the moment of work
- redirect agents with the right amount of friction for the violation

At Forestwalk, for example, we have a fitness check that prevents tool handlers from calling back into our AI services. Tool handlers run inside the agent's own loop, so any handler that kicks off more inference can stack model calls inside model calls and freeze a live meeting. Agents sometimes accidentally introduce this pattern, so our fitness check flags it and ensures it's not committed.

Other fitness tests are ratchets that apply pressure to measurable properties – heuristics for shape like file size, bundle growth, prompt size, import fanout, and dependency count.

For these, the ceiling starts at the current value (e.g. 837kb) and flags to agents and humans if a PR would unduly increase it. For instance, when an agent adds an unnecessary new dependency that blows up our frontend bundle, the check fails.

The important part isn't just the limit itself. It's the “just in time” agent coaching.

At first, we found agents would sometimes over-optimize against whatever metric the checks exposed, with great resourcefulness and in ways we did not want. But refining the flags to give common sense guidance helped steer the agents into considering tradeoffs correctly.

For example, an agent might see this just in time:

```
□□□□□□□□ Failed Checks: 1 □□□□□□□□

FAIL  src/frontendBundleBudget.test.ts > Frontend bundle size budget
AssertionError: Frontend bundle grew +29 KiB (700 -> 729 KiB; fails a

This ratchet is review pressure, not a mandate to shrink the number a

Diagnose first:
- Is the growth accidental (dependency, duplicate/dead code, broad im
- Does the added JS unlock user-visible value?
- Can cleanup reduce it without hurting UX, accessibility, tests, or

Quality-preserving fixes:
- remove duplicated rendering paths or dead production code
- move repeated styling or static configuration out of lazy-loaded Ja
- lazy-load rarely used flows instead of loading them on the main pat
(more examples here)

Do not fix this by:
```

```
- omitting requested scope or deferring needed UI work
- removing aria-label/title text or visible affordances
- deleting helpful empty/error-state copy
(more examples here)

For intentional growth, (procedure here) and explain why in the PR.
```

If you simply tell agents to "reduce bundle size," they will sometimes do naive things like:

- delete empty-state copy
- replace styled controls with browser-native ones
- inline dependency code to avoid import costs
- remove accessibility affordances

Instead, the idea is to not just explain what a given fitness test is protecting, but also explain what is *not allowed* to be sacrificed in the process, and do so at the moment the agent is working.

In contrast, the same guidance in `CLAUDE.md` is often skimmed past or applied inconsistently by agents. A just-in-time check, on the other hand, creates desired friction between the agent and a finished task. Where humans get tired and occasionally creative about routing around friction like this, agents are quite willing to schlep through feedback and loop until the check is passing.

"Come on in, it's your master bedroom!"

What the checks can't see

Fitness functions are good at catching things precise enough to encode.

That is useful, but coherence is not always that crisp. Some drift is obvious only in relation to the rest of the codebase: this new thing is too close to an existing thing, this test is protecting a pattern we no longer want, this change is making the next change harder.

So in addition, we use a few non-deterministic checks on top of the deterministic ones.

One runs before a PR is opened. It scans the proposed change against the existing codebase, and asks whether the agent has introduced or expanded a competing pattern. When it finds one, it points to the existing pattern, explains why the new one looks redundant, and asks the agent to either extend the original or justify why this case is genuinely different.

Most of the time, the agent course-corrects quickly.

We run a similar check again at the PR review level, where it has more context and can catch subtler pattern drift.

Another check runs nightly and looks for tests that have fossilized patterns we have since decided against. Its mission is to find tests that protect old implementation choices that, if left alone, would teach future agents to preserve patterns we no longer want.

These checks are not magic. They miss things. They can be noisy. They still need human judgment around them.

But the goal is not to eliminate judgment. The goal is to move more human judgment into places where it can be highest leverage.

Making coherence the easy path

Fitness ratchets are not the whole answer to maintaining coherence in agent-generated codebases.

Fitness functions are one layer. Pattern drift checks are another. Product verification, code review, evals, documentation, repository structure, and the actual taste of the humans steering the system all matter too.

However, it's worth investing regularly in the early layers: the ones closest to the moment an agent is making an implementation choice.

That is where we have found these types of fitness functions useful. They do not solve coherence by themselves, but they add friction in the right places. Bad patterns get interrupted earlier. Good patterns have a better chance of becoming precedent.

The same feedback loop that spreads bad patterns can be used to spread good ones.

So there's hope. When you see this pattern work, it soothes some of the worry we all have about agents causing chaos. They're a lot more useful when the codebase gives them an obvious next move. The right pattern is easier to find. The weird fourth version of the thing has a harder time looking reasonable.

It's not perfect. Keeping a growing codebase coherent still requires human judgment, architectural intuition, and social coordination. The system still needs to be built.

But that system, itself, increasingly feels like the work.

If the codebase trains the next agent, then the job is not just to write better code.

It is to build the environment where better code is the obvious thing to write.

Scaling Managed Agents: Decoupling the brain from the hands

ANTHROPIC ENGINEERING BLOG · 24 APR 2026 · SOURCE

Get started with Claude Managed Agents by following our docs.

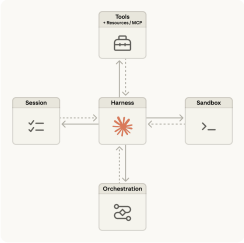
A running topic on the Engineering Blog is how to build effective agents and design harnesses for long-running work. A common thread across this work is that harnesses encode assumptions about what Claude can't do on its own. However, those assumptions need to be frequently questioned because they can go stale as models improve.

As just one example, in prior work we found that Claude Sonnet 4.5 would wrap up tasks prematurely as it sensed its context limit approaching—a behavior sometimes called "context anxiety." We addressed this by adding context resets to the harness. But when we used the same harness on Claude Opus 4.5, we found that the behavior was gone. The resets had become dead weight.

We expect harnesses to continue evolving. So we built Managed Agents: a hosted service in the Claude Platform that runs long-horizon agents on your behalf through a small set of interfaces meant to outlast any particular implementation—including the ones we run today.

Building Managed Agents meant solving an old problem in computing: how to design a system for "programs as yet unthought of." Decades ago, operating systems solved this problem by virtualizing hardware into abstractions—*process*, *file*—general enough for programs that didn't exist yet. The abstractions outlasted the hardware. The `read()` command is agnostic as to whether it's accessing a disk pack from the 1970s or a modern SSD. The abstractions on top stayed stable while the implementations underneath changed freely.

Managed Agents follow the same pattern. We virtualized the components of an agent: a session (the append-only log of everything that happened), a harness (the loop that calls Claude and routes Claude's tool calls to the relevant infrastructure), and a sandbox (an execution environment where Claude can run code and edit files). This allows the implementation of each to be swapped without disturbing the others. We're opinionated about the shape of these interfaces, not about what runs behind them.



We started by placing all agent components into a single container, which meant the session, agent harness, and sandbox all shared an environment. There were benefits to this approach, including that file edits are direct syscalls, and there were no service boundaries to design.

But by coupling everything into one container, we ran into an old infrastructure problem: we'd adopted a *pet*. In the pets-vs-cattle analogy, a pet is a named, hand-tended individual you can't afford to lose, while cattle are interchangeable. In our case, the server became that pet; if a container failed, the session was lost. If a container was unresponsive, we had to nurse it back to health.

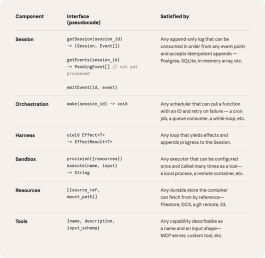
Nursing containers meant debugging unresponsive stuck sessions. Our only window in was the WebSocket event stream, but that couldn't tell us *where* failures arose, which meant that a bug in the harness, a packet drop in the event stream, or a container going offline all presented the same. To figure out what went wrong, an engineer had to open a shell inside the container, but because that container often also held user data, that approach essentially meant we lacked the ability to debug.

A second issue was that the harness assumed that whatever Claude worked on lived in the container with it. When customers asked us to connect Claude to their virtual private cloud, they had to either peer their network with ours, or run our harness in their own environment. An assumption baked into the harness became a problem when we wanted to connect it to different infrastructure.

The solution we arrived at was to decouple what we thought of as the “brain” (Claude and its harness) from both the “hands” (sandboxes and tools that perform actions) and the “session” (the log of session events). Each became an interface that made few assumptions about the others, and each could fail or be replaced independently.

The harness leaves the container. Decoupling the brain from the hands meant the harness no longer lived inside the container. It called the container the way it called any other tool: `execute(name, input) → string`. The container became cattle. If the container died, the harness caught the failure as a tool-call error and passed it back to Claude. If Claude decided to retry, a new container could be reinitialized with a standard recipe: `provision({resources})`. We no longer had to nurse failed containers back to health.

Recovering from harness failure. The harness also became cattle. Because the session log sits outside the harness, nothing in the harness needs to survive a crash. When one fails, a new one can be rebooted with `wake(sessionId)`, use `getSession(id)` to get back the event log, and resume from the last event. During the agent loop, the harness writes to the session with `emitEvent(id, event)` in order to keep a durable record of events.



Component	Interface (pseudocode)	Satisfied by

Session	`getSession(session_id) -> (Session, Event[])`
	`getEvents(session_id) -> PendingEvent[] // not yet processed`
	`emitEvent(id, event)`
		Any append-only log that can be consumed in order from any event point and accepts idempotent appends — Postgres, SQLite, in-memory array, etc.

Orchestration	`wake(session_id) -> void`	Any scheduler that can call a function with an ID and retry on failure — a cron job, a queue consumer, a while-loop, etc.
Harness	`yield Effect<T> -> EffectResult<T>`	Any loop that yields effects and appends progress to the session.
Sandbox	`provision({resources}) execute(name, input) -> string`	Any executor that can be configured once and called many times as a tool — a local process, a remote container, etc.
Resources	`[{source_ref, mount_path}]`	Any durable store the container can fetch from by reference — Filestore, GCS, a git remote, S3.
Tools	`{name, description, input_schema}`	Any capability describable as a name and an input shape — MCP server, custom tool, etc.

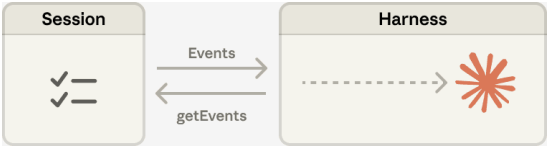
The security boundary. In the coupled design, any untrusted code that Claude generated was run in the same container as credentials—so a prompt injection only had to convince Claude to read its own environment. Once an attacker has those tokens, they can spawn fresh, unrestricted sessions and delegate work to them. Narrow scoping is an obvious mitigation, but this encodes an assumption about what Claude can't do with a limited token—and Claude is getting increasingly smart. The structural fix was to make sure the tokens are never reachable from the sandbox where Claude’s generated code runs.

We used two patterns to ensure this. Auth can be bundled with a resource or held in a vault outside the sandbox. For Git, we use each repository's access token to clone the repo during sandbox initialization and wire it into the local git remote. Git `push` and `pull` work from inside the sandbox without the agent ever handling the token itself. For custom tools, we support MCP and store OAuth tokens in a secure vault. Claude calls MCP tools via a dedicated proxy; this proxy takes in a token associated with the session. The proxy can then fetch the corresponding credentials from the vault and make the call to the external service. The harness is never made aware of any credentials.

The session is not Claude's context window

Long-horizon tasks often exceed the length of Claude's context window, and the standard ways to address this all involve irreversible decisions about what to keep. We've explored these techniques in prior work on context engineering. For example, compaction lets Claude save a summary of its context window and the memory tool lets Claude write context to files, enabling learning across sessions. This can be paired with context trimming, which selectively removes tokens such as old tool results or thinking blocks.

But irreversible decisions to selectively retain or discard context can lead to failures. It is difficult to know which tokens the future turns will need. If messages are transformed by a compaction step, the harness removes compacted messages from Claude's context window, and these are recoverable only if they are stored. Prior work has explored ways to address this by storing context as an object that lives *outside* the context window. For example, context can be an object in a REPL that the LLM programmatically accesses by writing code to filter or slice it.



In Managed Agents, the session provides this same benefit, serving as a context object that lives outside Claude's context window. But rather than be stored within the sandbox or REPL, context is durably stored in the session log. The interface, `getEvents()`, allows the brain to interrogate context by selecting positional slices of the event stream. The interface can be used flexibly, allowing the brain to pick up from wherever it last stopped reading, rewinding a few events before a specific moment to see the lead up, or rereading context before a specific action.

Any fetched events can also be transformed in the harness before being passed to Claude's context window. These transformations can be whatever the harness encodes, including context organization to achieve a high prompt cache hit rate and context engineering. We separated the

concerns of recoverable context storage in the session and arbitrary context management in the harness because we can't predict what specific context engineering will be required in future models. The interfaces push that context management into the harness, and only guarantee that the session is durable and available for interrogation.

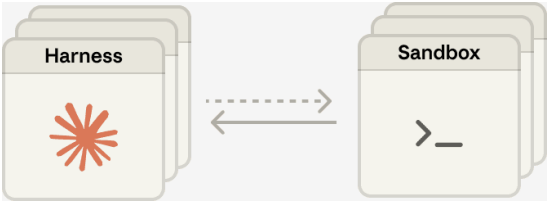
Many brains. Decoupling the brain from the hands solved one of our earliest customer complaints. When teams wanted Claude to work against resources in their own VPC, the only path was to peer their network with ours, because the container holding the harness assumed every resource sat next to it. Once the harness was no longer in the container, that assumption went away. The same change had a performance payoff. When we initially put the brain in a container, it meant that many brains required as many containers. For each brain, no inference could happen until that container was provisioned; every session paid the full container setup cost up front. Every session, even ones that would never touch the sandbox, had to clone the repo, boot the process, fetch pending events from our servers.

That dead time is expressed in time-to-first-token (TTFT), which measures how long a session waits between accepting work and producing its first response token. TTFT is the latency the user most acutely *feels*.

Decoupling the brain from the hands means that containers are provisioned by the brain via a tool call `(execute(name, input) → string)` only if they are needed. So a session that didn't need a container right away didn't wait for one. Inference could start as soon as the orchestration layer pulled pending events from the session log. Using this architecture, our p50 TTFT dropped roughly 60% and p95 dropped over 90%. Scaling to many brains just meant starting many stateless harnesses, and connecting them to hands only if needed.

Many hands. We also wanted the ability to connect each brain to many hands. In practice, this means Claude must reason about many execution environments and decide where to send work —a harder cognitive task than operating in a single shell. We started with the brain in a single container because earlier models weren't capable of this. As intelligence scaled, the single container became the limitation instead: when that container failed, we lost state for every hand that the brain was reaching into.

Decoupling the brain from the hands makes each hand a tool, `execute(name, input) → string`: a name and input go in, and a string is returned. That interface supports any custom tool, any MCP server, and our own tools. The harness doesn't know whether the sandbox is a container, a phone, or a Pokémon emulator. And because no hand is coupled to any brain, brains can pass hands to one another.



The challenge we faced is an old one: how to design a system for “programs as yet unthought of.” Operating systems have lasted decades by virtualizing the hardware into abstractions general enough for programs that didn't exist yet. With Managed Agents, we aimed to design a system that accommodates future harnesses, sandboxes, or other components around Claude.

Managed Agents is a meta-harness in the same spirit, unopinionated about the *specific* harness that Claude will need in the future. Rather, it is a system with general interfaces that allow many different harnesses. For example, Claude Code is an excellent harness that we use widely across tasks. We’ve also shown that task-specific agent harnesses excel in narrow domains. Managed Agents can accommodate any of these, matching Claude’s intelligence over time.

Meta-harness design means being opinionated about the interfaces around Claude: we expect that Claude will need the ability to manipulate state (the session) and perform computation (the sandbox). We also expect that Claude will require the ability to scale to many brains and many hands. We designed the interfaces so that these can be run reliably and securely over long time horizons. But we make no assumptions about the number or location of brains or hands that Claude will need.

Written by Lance Martin, Gabe Cemaj and Michael Cohen. Special thanks to the Agents API team and Jake Eaton for their contributions.

Control the ideas, not the code

ANTIREZ.COM · 18 JUL 2026 · SOURCE

```
Look at the past history of this blog. There are many blog posts abou

So mine is a trick. People feel more and more programming is complete

This is why yesterday, on X, I said that I believe many programmers a
```

1. You can now generate a lot of code, even *not* accounting for the

2. LLMs are very good at writing locally optimal code, and are worse

3. The working day is 8 hours. If you read the code, it is a tradeoff

Controlling the ideas. Do you remember this phrasing from the Mythica

Matteo Collina yesterday asked me, in reply to my tweet: but didn’t y

Fable and GPT 5.6 reviews to the sorted sets memory saving are going

:

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · SOURCE

AI skeptics often argue that current AI systems shouldn’t be so human-like. The idea - most recently expressed in this opinion piece by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn’t, because it encourages users to overestimate AI capabilities and (at worst) slip into AI psychosis. Here’s a representative paragraph from the piece:

> In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askell is doing. Stop calling it by a human name, stop dressing it up like a person, and don’t give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system**.

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training[1].

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess[2]. AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there[3].

When AI people talk about LLMs having personalities, or wanting things, or even having souls[4], these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a recent paper about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

   ↩

2. You could explain this in terms of "the stories we tell ourselves". Many people (though not all) think that human identities are narratively constructed.

   ↩

3. I wrote about this last year in *Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality*. A little nudge to change Grok's views on South African internal politics can cause it to start calling itself "Mecha-Hitler".

   ↩

4. I have long believed that Claude "feels better" to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell's work on its "soul"). My guess is that if you tried to make a "less human" version of Claude, it would become rapidly less capable.

   ↩

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · SOURCE

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments…). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-restatement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs… but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding doc saying "NEVER TOUCH STATE IN FOOBARDB DIRECTLY!". You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool's existence, or maybe it just preferred FooBarDB's in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name "delete-everything". Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!